

ECE 622: Modeling & Verification of Embedded Systems

Spring 2026: Lab 3: Hybrid Systems Verification

Prof. Daniel Holcomb

Submitted by:	Jaideep Khare and Shu-Lin Shih
Student ID:	35193853 and 35199856
Lab No.:	3
Submission Date:	April 14, 2026 (04/14/26)

Overview

In this lab we used **PHAVerLite** to compute forward reachable sets of hybrid automata and to check safety-style properties. For each problem we encoded locations with continuous dynamics (written in PHAVer syntax using primed variables, e.g. $\mathbf{x}' == \mathbf{v}$ for $\dot{x} = v$) and discrete jumps with guards/resets, then bounded the analysis using reasonable location invariants. We varied the partition constraint (**pc**) to study the tradeoff between runtime and precision: smaller **pc** refines the polyhedral over-approximation and can remove spurious reachability, but increases the number of symbolic locations and CPU time. Throughout, we treated results in the standard soundness direction for over-approximation [Frehse, 2008]: “**not reachable**” for a bad set is a proof for the modeled system, whereas “**reachable**” indicates the property is not proven at that abstraction level and may require refinement. To keep experiments reproducible, long runs were capped with a fixed timeout and recorded as timed-out when necessary.

1 Part A: Analysis of Bouncing Ball Model

The bouncing-ball experiment is the canonical hybrid model (continuous flight with gravity, discrete bounce resets). Course notes `src/module3_hybrid_timed.pdf` slides 92–94 illustrate the same formal pattern (locations, flows, guards, resets) used when encoding the ball in PHAVerLite; the lab model instantiates that template for reachability experiments below.

A.1: Reachable sets for two values of **pc**

Use *PHAVerLite* to compute reachable states with **pc**=0.5 and **pc**=0.05. Plot both on the same axes (clearly distinguishable). Answer: for each **pc**, is $(x, v) = (0.55, 0.0)$ unreachable based on the plot?

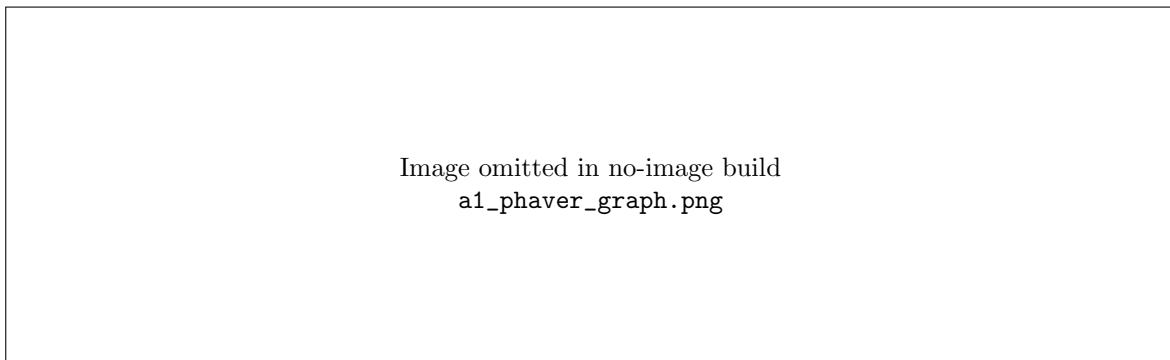


Figure 1: A.1: Reachable-state plot used to classify the target $(x, v) = (0.55, 0.0)$ at two partition settings.

We first used the reachable-set plot as a coarse check of whether the target point is plausibly excluded by the reachable enclosure. At the coarser partition (**pc**=0.5), the enclosure is visibly wider and the target point is not excluded by the plot. After refining to **pc**=0.05, the enclosure tightens and the target point is visually outside the reachable region.

Final answer (A.1): at **pc**=0.5 the point is **not shown unreachable** by the plot; at **pc**=0.05 it is **unreachable** by the plot.

A.2: is_reachable vs. plot; sweep pc

Verify $(0.55, 0.0)$ with `is_reachable` for $pc=0.5$ and 0.05 (consistency with A.1). Then sweep pc over $0.5, 0.2, 0.1, 0.05, 0.01$, and 0.005 . If any run exceeds 15 minutes, report time as “timed out.”

Example command form (adjust names to your model):

```
check = bball.is_reachable(target);
```

Partition pc	Condition reachable?	Num locations reached	CPU time (s)
0.5	yes	33	0.03
0.2	yes	71	0.077
0.1	yes	152	0.171
0.05	no	327	0.439
0.01	no	2671	8.814
0.005	no	5373	44.513

Table 1: A.2 reachability sweep from PHAVerLite outputs in `src/lab3.html`.

Follow-up (no extra runs). If you repeated the reachability check for $pc=1.0$, $pc=0.075$, and $pc=0.001$, which case(s) would *definitely* report the target unreachable? Answer from soundness of unreachability under over-approximation and the table above.

The sweep shows the expected precision-runtime tradeoff: as pc decreases, the number of symbolic locations and CPU time increase sharply, but the reachable enclosure becomes tighter. In our runs the reachability verdict flips between $pc=0.1$ (reachable) and $pc=0.05$ (not reachable), which is consistent with refinement removing spurious reachability.

Final answer (A.2 follow-up): among $pc=1.0$, 0.075 , and 0.001 , the case most likely to *definitely* report the target unreachable is $pc=0.001$ (finest refinement).

A.3: Bounce counter n and peak-height queries

Fix $pc=0.05$. Discuss whether peak height exactly 0.15 on bounces 1–4 appears consistent with the reachable set plot. Add variable n (init, derivatives per location, resets on jumps). Enforce $n \leq 10$ via an invariant. Add four `is_reachable` checks $n1 \dots n4$ for $(x, v) = (0.15, 0)$ on each bounce. Submit `<lastname>_A3.pha`.

Image omitted in no-image build
a3_phaver_graph.jpg

Figure 2: A.3: Reachable-set plot used to inspect whether $(x, v) = (0.15, 0)$ aligns with bounce peaks.

```
check = bball.is_reachable(n1);  
check = bball.is_reachable(n2);  
check = bball.is_reachable(n3);  
check = bball.is_reachable(n4);
```

Image omitted in no-image build
a3_terminal_output.jpg

Figure 3: A.3: Terminal output for bounce-indexed queries **n1–n4**.

In A.3 we used two complementary checks: a geometric inspection of the reachable enclosure to see whether $(0.15, 0)$ lies on a plausible peak, and a direct reachability query indexed by bounce count. Both checks point to the same conclusion: the second-bounce peak is consistent with height 0.15 at zero velocity, while the other peaks do not appear supported at this partition.

Final answer (A.3): height 0.15 at the peak is consistent with the **2nd bounce** (at $pc=0.05$), matching both the plot and query output.

2 Part B: Model of Heating System

The heating plant is a two-location hybrid: temperature evolves continuously in each mode while a clock t advances at rate one, so time-bounded properties can be expressed as conjunctions involving t . Location invariants (e.g. $0 \leq t \leq 10$) keep the reachable set compact for PHAVerLite.

B.1: Reachable set with $pc=0.3$

Encode the heating model in PHAVer syntax. Add a clock t (init 0, derivative 1 in both locations, value retained across jumps). Enforce $0 \leq t \leq 10$ in both locations. Plot reachable states: x -axis time, y -axis temperature.

Image omitted in no-image build
b1_phaver_state_graph.png

Figure 4: B.1: Heating-system reachable set (time on x -axis, temperature on y -axis) at $pc=0.3$.

We implemented the clocked hybrid model and plotted the reachable set in the time–temperature plane. The resulting tube provides a compact summary of the system’s possible temperature evolution over the bounded horizon, and it makes the effect of switching between modes visible as changes in the reachable enclosure.

Final answer (B.1): the time–temperature reachable set was generated at $pc=0.3$ for $t \in [0, 10]$.

B.2: Time-bounded temperature property

From initial (cool, $t=0, x=22$), show temperature goes below 20°C within 0.98 s. Use a counterexample region with $t==0.98$ (not $t>=0.98$). Find the largest pc (within 0.1 of the true threshold) for which the bad set is unreachable. Submit `<lastname>.B2.pha`.

We expressed the requirement as a bad set at the exact time slice $t = 0.98$ and then swept pc to find where the abstraction becomes too coarse to exclude the bad region. In our sweep, the check remained *not reachable* below $pc=1.3$ and became *reachable* at $pc=1.3$, giving a tight bracket for the requested threshold.

```
bad = heatingsystem.{ $ & t == 98/100 & x > 20 };
check = heatingsystem.is_reachable(bad);
```

Final answer (B.2): the largest partition constraint that still reports the bad set **not reachable** is $pc=1.2$; at $pc=1.3$ the bad set becomes **reachable**.

3 Part C: Car Acceleration and Braking

Scenario: two intersections 500 m apart; stop sign then traffic light. Full stop at sign; accelerate at 3.5 m s^{-2} . Braking for red light starts between 250 m and 300 m past the stop sign (i.e. 200 m–250 m from the light). Braking acceleration -5.5 m s^{-2} until stop.

The red-light onset is modeled as **nondeterministic**: the jump from acceleration to braking may occur at any position in $[250 \text{ m}, 300 \text{ m}]$. The invariant $x \leq 300 \text{ m}$ in the accel location forces a decision before leaving that window.

C.1: Hybrid automaton diagram

Circle locations; label guards on jumps; derivatives of position and velocity per location; location invariants (including to force transitions when guards stay true).

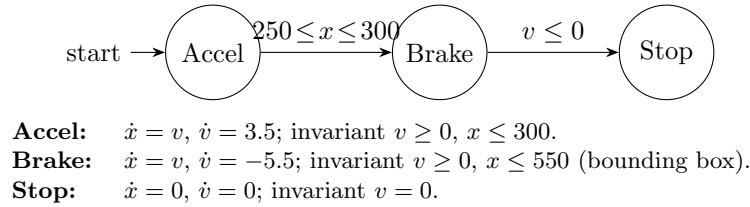


Figure 5: C.1: Hybrid automaton for the car (initial full stop at $x = 0$).

Analytical cross-check (kinematics)

Uniform acceleration segments satisfy the usual SUVAT relations; they are useful to sanity-check extreme speeds and stopping distances against the reachable set, not as a substitute for PHAVerLite proofs:

$$v^2 = u^2 + 2as, \quad (1)$$

$$s = ut + \frac{1}{2}at^2, \quad (2)$$

$$v = u + at. \quad (3)$$

For example, from rest with $a = 3.5 \text{ m/s}^2$ until $x = 300 \text{ m}$, (1) gives $v = \sqrt{2 \cdot 3.5 \cdot 300} \approx 45.8 \text{ m s}^{-1}$ if the vehicle could accelerate the full distance without braking—the model instead branches into braking in $[250, 300]$, so the tool envelope should reflect that nondeterminism.

C.2: PHAVer model and reachable set

Translate to PHAVer. Use reasonable invariants to bound the state space. With $pc=3.0$, plot reachable states: position (x -axis) vs. velocity (y -axis).

We encoded the car dynamics and red-light nondeterminism in PHAVerLite, bounded the state space with simple invariants, and generated the reachable set at $pc=3.0$. We then exported the reachable region and plotted position versus velocity to visualize the envelope of possible behaviors under the uncertain braking start.

Image omitted in no-image build
c2_car_reachable_x_v.jpg

Figure 6: C.2: Reachable states in the (x, v) plane at $pc=3.0$ (export from PHAVerLite).

Final answer (C.2): reachable states in the (x, v) plane were generated at $pc=3.0$.

C.3: Three reachability properties

For each property below, give the exact `is_reachable(...)` query, include one screenshot with PHAVerLite text output for all three checks, and explain why unreachability (or the appropriate abstraction argument) proves the property true/false. Submit `<lastname>_C3.pha`.

Property	Summary
1	The car will definitely stop before it reaches the 2nd intersection.
2	The velocity cannot be above 46.5 ms^{-1} while braking.
3	The velocity cannot be above 40.0 ms^{-1} while braking.

Table 2: C.3 properties (from lab handout).

Bad-state queries (submitted jkhare_C3.pha). Each property is checked by reachability of a *violation* set; proving the property would require `not reachable` for that set.

```
p1_bad = car_sys.{Brake & x >= 500 & v >= 0.001};
p1_bad_reach = car_sys.is_reachable(p1_bad);
p2_bad = car_sys.{Brake & v >= 46.5};
p2_bad_reach = car_sys.is_reachable(p2_bad);
p3_bad = car_sys.{Brake & v >= 40.0};
p3_bad_reach = car_sys.is_reachable(p3_bad);
```

Here $v \geq 0.001$ in P1 separates “still moving” from a full stop at $v = 0$. At $pc=3.0$, PHAVerLite reported **reachable** for all three bad sets; therefore **none** of the three properties is formally proven as stated (unreachability would be required for a sound proof). Reachability here may be due to over-approximation, but it does not establish safety—tighter pc or alternative encodings would be needed to refine the verdict.

Listing 1: C.3: PHAVerLite output summary for the three property checks ($pc=3.0$).

```
p1_bad is reachable.
p2_bad is reachable.
p3_bad is reachable.
```

Final answer (C.3): at $pc=3.0$, all three bad sets were reported **reachable**, so Properties 1–3 are **not proven** under this abstraction.

C.4 (ECE 622 only): 0–60 MPH time estimate

Add a time variable; $60 \text{ mph} = 26.8 \text{ ms}^{-1}$. Use $pc=3.0$. Two queries: (i) 26.8 ms^{-1} unreachable by time T ; (ii) reachable by time $T + 0.1 \text{ s}$. Report an interval for 0–60 time. Submit `<lastname>_C4.pha`.

A separate automaton `car_c4` adds t with $\dot{t} = 1$ in the `Accel` location (constant acceleration from rest, $\dot{v} = 3.5$). Bracketing queries at $T = 7.6 \text{ s}$ and $T + 0.1 \text{ s} = 7.7 \text{ s}$:

```
q_before = car_c4.{Accel & t <= 7.6 & v >= 26.8};
r_before = car_c4.is_reachable(q_before);
q_after = car_c4.{Accel & t <= 7.7 & v >= 26.8};
r_after = car_c4.is_reachable(q_after);
```

With $pc=3.0$, the tool reported `q_before` not reachable and `q_after` reachable, so the 0–60 mph time lies in the interval $(7.6\text{ s}, 7.7\text{ s}]$ (consistent with $t = v/a$ for constant a at 26.8 m s^{-1}).

Final answer (C.4): 0–60 mph time is in $(7.6\text{ s}, 7.7\text{ s}]$.

Conclusion

Across Parts A and B we saw the central practical lesson of PHAVerLite reachability: decreasing pc substantially increases cost (locations and runtime) but can sharpen the reachable enclosure enough to change the reachability verdict for a boundary target. For the heating model, bracketing over pc provided a concrete way to locate where the abstraction stops being strong enough to exclude a time-slice counterexample. In Part C, we encoded environmental uncertainty as nondeterminism in the braking trigger and checked safety-style statements as bad-state reachability queries; the over-approximate semantics make *unreachability* the reliable proof direction. In our runs, the three C.3 bad sets were reachable at $pc=3.0$ (so not proven), while the C.4 bracketing approach produced a narrow and interpretable 0–60 mph time interval.

Appendix: Tooling and submission artifacts

This report template was adapted from the Lab 2 `latex-build` layout. Build with `latexmk` (see `README.md`). PHAVerLite models and plots were produced on the course toolchain; figures can be synced via Git alongside this `.tex` file.

Reproducibility (Part C). Scripts under `lab3/modeling_car/`: `run_c3.sh` (timeout-wrapped PHAVerLite), `check_c3_properties.sh`, `plot_c3.sh` for the (x, v) figure; `lab3/modeling_car/c4/run_c4.sh` for C.4 bracketing runs.

Files to submit (replace `<lastname>` per handout):

- `<lastname>_A3.pha`
- `<lastname>_B2.pha`
- `<lastname>_C3.pha`
- `<lastname>_C4.pha` (ECE 622 only)

References

Goran Frehse. PHAVer: Algorithmic verification of hybrid systems past HyTech. *International Journal on Software Tools for Technology Transfer*, 10(3):263–279, 2008. doi: 10.1007/s10009-007-0062-3.